ELI — Inference And Hardware

ELI v2.5.31

November 2026

Contents

ELI Inference & Hardware Boot **1**

Inference (cognition/gguf_inference.py, 3.2k LOC + inference_broker.py) . . . 1
Hardware profiling (core/hardware_profile.py, ~3200 LOC) . . . 2
GPU pack (packaging/pyinstaller/eli_gpu_pack.py, core/gpu_pack_runtime.py) . . . 4
Load plan (core/load_plan.py) . . . 4
Settings (core/runtime_settings.py, ~1150 LOC) . . . 5
Paths (core/paths.py, 808 LOC) . . . 6
STT VRAM awareness . . . 6
Piper and LoRA device selection . . . 6
Honest assessment . . . 6

ELI Inference & Hardware Boot

> **Updated for v2.5.21.** GGUF path is canonical (optional Ollama backend still exists as a secondary path). What a model loads with is planned by llama.cpp's own load simulation on every GPU it sees (core/load_plan.py); the prompt format is the model's own chat template. Token budgets scale by reasoning mode via reasoning_modes.py.

How ELI loads a model, talks to it, and adapts to whatever machine it's on. The inference path is model-agnostic (see memory eli-model-agnostic); the boot path is hardware-adaptive. Files in eli/cognition/ and eli/core/.

Inference (cognition/gguf_inference.py, 3.2k LOC + inference_broker.py)

- **Model resolution (get_model_path)**: ELI_GGUF_MODEL_PATH env → model_path/custom_model_path/bundled_model_p settings keys. No baked model; empty default.
- **load_model(force_reload)**: load_request() reads the settings (env first): context (0 = auto), threads (0 = one per physical core), batch, cache type, GPUs to use and their shares, and a GPU-layer count only when it was pinned for this model at this context (pinned_gpu_layers_for_model; -1, llama.cpp's own value, is every layer). load_plan.plan() turns that into the load (below) and _build_with_plan makes it. A load that still fails is planned again for the memory free at that moment, then on the processor (three attempts); a cache type llama.cpp refuses is planned again with the default cache; a GPU pack too old for the model hands over to the bundled runtime. What was asked and what loaded are both written to the runtime snapshot, read from the live model.
- **Chat format from the model** (cognition/chat_template.py): every GGUF chat model carries the Jinja template it was trained with (tokenizer.chat_template); rendering it gives each model its own format, and rendering it around sentinel text gives its reasoning markers and end-of-turn marker. The family formats in _format_chat are kept only for a file with no template or one that will not render. Thinking is switched through the template's own enable_thinking where it has one.

- **Serialization**: all calls hold `_LLM_CALL_LOCK` (a native RLock from `runtime/native_locks`) — llama_cpp is not safe under concurrent calls; this is also what vision hot-swap and the ambient daemon coordinate on.
- **The model's state is kept at the end of a prompt's fixed part**: `_format_prompt` notes where the part that does not depend on the message ends (the system prompt, or the intent resolver's action list and examples). Before a call, `_resume_from_saved_state` gives back the state saved there by an earlier call that started the same way, or reads that part alone and saves the state (llama.cpp's per-sequence state calls: the model's memory only, no logits). The model then reads only what follows. A hybrid model (`qwen35moe`: most layers carry a running state, not an attention cache) cannot be rolled back to a shared prefix, so without this every call read its whole prompt: measured on Qwen3.6-35B-A3B, an RTX 2060 SUPER and experts in RAM, Normal mode's second pass took 63.8 s and took 3.2 s with the state given back (the same 60-token answer both ways); the intent resolver went from 26.6 s to 2.5 s after its first call. A 7k-token state is 135 MiB, saved in 0.08 s and restored in 0.02 s. The states are kept within an eighth of free memory and two contexts' worth of state, the oldest giving way; background work uses them but never adds one; they go when the model is unloaded. `ELI_PROMPT_STATE_REUSE=0` turns it off.
- **Prefill abort**: shutdown and cancel register `llama_set_abort_callback` so a long prompt prefill stops immediately — `StoppingCriteria` alone only runs between output tokens.
- **Live control**: `get_live_runtime_override`, `unload_model`, `reload_model` let the GUI swap models / change settings without a full restart.
- **`InferenceBroker`** (`inference_broker.py`): the higher-level `infer()` / `gguf_ready` abstraction the orchestrator, engine, and ReAct loop call, so callers don't touch `gguf_inference` directly.
- **CPU runtime preflight (`core/llama_cpu_compat.py`)**: before the first native load, a cached smoke test (once per llama.cpp build and CPU) checks the runtime starts on this CPU. Prebuilt wheels assume AVX-VNNI and SIGILL on older chips; the loader now reports that plainly instead of crashing. Only a definitive illegal-instruction blocks a load. `ELI_CPU_PREFLIGHT=0` disables it.

Hardware profiling (`core/hardware_profile.py`, ~3200 LOC)

Every GPU, every vendor, every OS, read by one function per source and shared by detection (`detect_hardware`) and live telemetry (`get_live_gpu_telemetry`, printed by `describe_gpus`):

Source	Cards	Where
`_nvidia_cards`	NVIDIA, per card	`nvidia-smi`, any OS
`_amd_cards`	AMD, discrete and APU	`amdgpu` kernel counters (no ROCm needed), else `rocm-smi`
`_windows_dxgi_adapters`	any vendor	Windows DXGI: memory and the budget Windows gives this process
`_macos_accelerators`	Apple Silicon, AMD on Intel Macs	`ioreg` IOAccelerator
`_linux_drm_gpus`	every display device once, PCI and SoC (msm, asahi)	`/sys/class/drm`
`_llama_cpp_cards`	whatever llama.cpp sees (CUDA, Vulkan, Metal, HIP, SYCL)	fills what a vendor tool leaves out, once llama.cpp is loaded

A model uses the discrete cards, or the integrated ones when there are none (llama.cpp's rule, `_cards_in_use`). Each card's maker comes from its llama.cpp backend or its name (`_vendor_of`).

- **No guessed figures.** A card whose memory no source reports (an NVIDIA driver whose `nvidia-smi` fails, Intel Arc on Linux, Windows before 10 for the free figure) keeps its name, its memory is left unknown and `gpu_detection_uncertain` is set; `effective_use_gpu_layers` still plans for it

and llama.cpp measures it before a load. The tables of NVIDIA and Arc models with their factory memory, the flat 4 GB, the "keep a fifth for the desktop" share and the RAM floors are gone.

- **Unified memory** (Apple Silicon, APUs, iGPUs): the same fraction of available RAM as `cpu_ram_budget_mb()`, with no floor or ceiling of ELI's own.
- Subprocess calls to `nvidia-smi` / `rocm-smi` / `lspci` sanitize `LD_LIBRARY_PATH` first (`_external_tool_env()`): a frozen build points it at its own bundle.
- **Threads**: `recommend_cpu_threads()` = the physical performance cores (measured on an i7-8700: 4.37 tokens/s at 6 threads, 3.99 at 12). The setting's 0 means this rule. A count the old tuner saved (logical threads less one or two) goes back to 0 once (`runtime_settings` revision 1, recorded as `settings_revision`, so a count set afterwards stays); read once per process (`physical_performance_cores` is cached; it was logged on every hardware check).
- **KV cache type** — the settings `cache_type_k` / `cache_type_v` are empty (fp16, the default) or `q8_0`. Any other quantized type stored there reads, sizes and loads as `q8_0` (`core/gguf_sizes.py`: `cache_setting`, `effective_cache_type`): measured on llama.cpp 0.3.35, a `q8_0` cache answered the same as fp16 at the same speed on CUDA, Vulkan and the processor, and a `q4_0` cache answered wrongly. The tuner chooses `q8_0` only when an fp16 cache would not hold the context in the memory left after the weights. llama.cpp is given the type as `type_k` / `type_v` with `flash_attn` (`llama_cache_args`); arguments it does not take are dropped and logged (`model_load_diagnostics.llama_accepts`), a cache it refuses is loaded again as the default with the fit redone, and the type it reports is what the runtime snapshot records.
- **The recommendation is the plan.** `recommend()` for one model asks the load planner (`_recommend_from_plan`), so the startup window shows what the loader will do. The older estimate (`smart_fit_config`, `unified_fit_config`, `_compute_graph_reserve_mb`, `_kv_cache_mb`) is used only where llama.cpp cannot simulate (no runtime, or several candidate models).
- **Fit profiles** — startup exposes **Balanced / Max GPU / Max context** (`fit_priority` in settings, `ELI_FIT_PRIORITY` env): Balanced sheds GPU layers → batch → ctx; Max GPU shrinks ctx/batch before dropping layers; Max context moves layers to the processor first. The planner searches in that order.

Install-time detection is a SEPARATE stack from `hardware_profile`

`hardware_profile` only runs once ELI itself is importable. Getting there — deciding whether to build `llama-cpp-python` with CUDA/ROCm/Vulkan in the first place — happens before that, in two more independent detectors that do not share its code at all: `install.sh` (bash, the actual build) and `eli/setup/hardware_policy.py` (Python, decides `ELI_INSTALL_CPU_ONLY` for `eli_setup.sh`'s GUI-wizard-or-terminal install). A bug fixed in one of the three does nothing for the other two — confirmed in the field, twice:

- **Both install-time detectors misread nvidia-smi's specific NVML "Driver/library version mismatch" failure as a working GPU.** That failure prints a first line matching the usual "broken driver" keyword filters ("Failed to initialize NVML: …") *and* a second line that matches none of them — "NVML library version: 595.91" — which both `install.sh`'s bash filter and `hardware_policy._nvidia_smi_gpu_names()` were reading as a real GPU product name. Chosing the GPU build plan on a driver that cannot currently build or run CUDA is what sent one field install into a doomed from-source CUDA build that failed partway through — which is what "the GUI wizard did not finish" actually was; the wizard has no build timeout of its own, `install.sh` really did exit non-zero. Fixed by checking nvidia-smi's *exit code* first in both places (reliable on both failure shapes) rather than only pattern-matching its stdout text.
- **`hardware_policy.py` also treated a bare `rocm-smi` binary as a working AMD GPU** — installed as part of a generic driver-utils package with no AMD hardware behind it, it still errors ("Driver not initialized (amdgpu not found in modules)") the moment it is actually queried. Fixed to require a real product-name answer, not just the binary existing.
- `install.sh`'s from-source CUDA build also used to fall back to `CMAKE_CUDA_ARCHITECTURES=native` when nvidia-smi's `compute_cap` query came back empty — but "native" makes `nvcc` re-probe the GPU through the same driver stack nvidia-smi just failed to reach, so it inherits the exact same failure. Falls back to the same known-good architecture list the CI-built GPU pack uses (`61;75;86;89`,

Pascal–Ada) instead.
- **Known gap, not yet closed**: even after these fixes, a portable install that correctly lands on `--cpu-only` (driver unverifiable, or a genuinely weak/absent GPU) has no path back to GPU acceleration *during* that install — `install.sh` and `eli_setup.sh` have no awareness of the CI-built `gpu-packs` release at all; only the runtime "Install CUDA/Vulkan GPU pack" button in the Startup Model Selection dialog does. `install.sh` now at least tells the user that button exists instead of going quiet about GPU acceleration entirely. Teaching the portable installer to try the GPU pack itself (matching what the frozen/AppImage runtime already does automatically) is the real fix and is still open.

GPU pack (`packaging/pyinstaller/eli_gpu_pack.py`, `core/gpu_pack_runtime.py`)

Portable/AppImage builds download a CUDA or Vulkan build of `llama-cpp-python` per machine (the bundled runtime is CPU-only, safe everywhere) and shadow the bundled copy via `sys.path`.

- **`gpu_pack_looks_installed()`** is the trust-first check every normal boot uses: marker file + backend metadata + native libs present on disk, no subprocess, no GPU/driver contact. Only when this can't confirm the pack does anything fall through to `gpu_pack_operational()`, which live-probes offload in a throwaway subprocess — correct at install time, but a transient driver hiccup or busy GPU used to make that live probe fail even with a perfectly good pack installed, and any caller treating a failed live probe as "not installed" deleted a working pack and re-downloaded it on the very next launch. The cheap check is what stops that loop.
- **`activate_gpu_pack_runtime()`** preloads native libs, purges cached `llama_cpp`, calls `llama_backend_init()`, and verifies offload in-process — so a pack that verified in a subprocess actually works in the AppImage GUI process (Intel Iris Xe Vulkan is the case that needed this).
- `try_activate_gpu_pack()` (`gpu_pack_runtime.py`) delegates entirely to `activate_gpu_pack_runtime()`'s own (correctly cheap-first) gating rather than duplicating a check — a duplicate with reversed operand order here used to force the subprocess probe on every single GUI launch regardless of whether the marker file already said the pack was good.
- Writes `runtime/gpu/.gpu_pack_ok` + `.gpu_pack.json` (backend, version) on success; respects `runtime/.gpu_choice` for CPU opt-out.
- **One source for the runtime**: the bundled `llama-cpp-python` and every pack are built from `packaging/llama_source.py`: pinned llama-cpp-python and llama.cpp archives checked against their SHA-256, the binding changes a newer llama.cpp needs, and the version in `packaging/llama_cpp_constraints.txt` (a label of its own, so installed packs are replaced when it moves). A pack of a different build than the llama.cpp already running in the process is left for the next start: the two share library names, and two builds in one process freed each other's memory.
- **One llama.cpp per build**: a pack is used only at the bundled runtime's exact version (`pack_matches_runtime`). When an update moves llama.cpp on, `outdated_pack()` names the old pack and the first-run chooser (`eli_entry._first_run_gpu_offer`) replaces it with the matching one, same backend, without asking again: the GPU was chosen already. Every download reports its real size to the progress window (`install(argv, progress)`); the startup window and the first-run wizard share one status reader (`startup._gpu_pack_state`) and offer "Update"; server and helper starts print how to update.
- **Backend** follows the detected vendor (`pack_backend_for`): CUDA on an NVIDIA card, Vulkan on every other GPU.
- **Size on disk**: a wheel cannot hold links, so every soname chain arrives as full copies (`libggml-cuda.so`, `.so.0`, `.so.0.25.3`: three 498 MB files). `_dedupe_wheel_native_trees` links identical names to the versioned file right after unpacking, before the CUDA runtime is fetched, and each wheel is deleted once unpacked: the 0.3.36 CUDA pack takes 1.1 GB instead of 2.0 GB, and the install needs that much less free space (2.5.18 failed on a full disk at the cuBLAS step).

Load plan (`core/load_plan.py`)

llama.cpp can simulate a load (`no_alloc`): it builds the model and a context the way a real load would and, allocating nothing, logs which device each layer and cache goes to and how large every compute buffer is. The planner asks it about the operator's settings and, when they do not fit what each GPU has free

now, about smaller ones in the fit profile's order, and keeps the largest that fits every device. It replaced a ladder of guessed configurations, a load probe whose checks timed out (30 of 32 since September) and a one-card fit.

- **Several GPUs**: layers are shared so each card fills the same share of what it holds after its compute buffers and headroom (`tensor_split`); the startup window chooses which cards and, optionally, manual shares (`gpu_devices`, `tensor_split`). `CUDA_DEVICE_ORDER=PCI_BUS_ID` keeps llama.cpp, nvidia-smi and the settings numbering cards the same way. A manual split shares layers, and the experts kept in RAM are the first layers': when that leaves a card with room for another layer's experts, the plan says so and that Auto fills each card (two cards at 50/50: 1.4 GB used of 6 GB free on the first while the experts of 29 layers were in RAM). The status line names every card in use ("RTX 2060 SUPER ×2"), and the tuning panel shows "loaded" only when a model is.
- **Headroom**: what llama.cpp takes on a card after the load as it works through the first prompts, measured on this machine and kept per card and model in `data_dir/runtime/gpu_headroom.json` (`watch_headroom`, `note_headroom`). Until a card is measured the plan keeps llama.cpp's own compute-buffer figure free on it. `ELI_VRAM_RESERVE_MB` is used as given. Measured on two RTX 2060 SUPER: 28-82 MB, where a fixed 700 MB was held back.
- **Mixture-of-experts**: a model that does not fit keeps the experts of its first layers in RAM, as few as the GPUs need (`tensor_buft_overrides`).
- **Processor only**: the largest context up to the target whose weights, cache and buffers fit the RAM budget, from two simulations (the cache grows with the context) and a third to confirm.
- **No simulation** (an older llama.cpp): the same search on the file's own tensor sizes (`core/gguf_sizes.py`) with the buffers estimated, and the plan says so. A file whose tensor table cannot be read is loaded as asked (`as_asked`).

`runtime_load_gap()` (`cognition/context_synthesiser.py`; -1 asked and none loaded is a reduction) only ever checked for `effective < requested` ("reduced," the classic VRAM-shortfall case) — a mixture-of-experts load *raises* `n_gpu_layers` above the request instead, and that case reported "loaded exactly as requested" for a load that did not match the request at all. It now reports a separate `raised` dict, and every surface that reads it (`GPU_STATUS`, `EXPLAIN_COGNITION_RUNTIME`'s `_inference_runtime_lines`, and `live_runtime_brief()` which grounds casual/phatic replies) states plainly which layers' attention/shared weights are on the GPU, how much VRAM that is, how many GB of experts sit in RAM, and that the CPU processes those during generation — not "the rest run on CPU" (backwards under MoE: nothing was left off the GPU) and not silence. `live_runtime_brief()` returning "" for a MoE load specifically is why a phatic reply once stated a stale, invented layer count: the grounding it would have drawn from was empty.

Settings (`core/runtime_settings.py`, ~1150 LOC)

- `DEFAULTS` (the full settings schema) + `ENV_TO_KEY` (env-var overrides).
- `load_settings` / `save_settings` / `update_settings`.
- **Redistribution-aware**: `_migrate_legacy_keys` (schema evolution), `_resolve_relative_model_paths` + `_heal_model_paths` (fix stale absolute paths when the project moves machines), and `_portable_settings_for_sto` (strip machine-specific values before storage) — though personal values like `user_name` can still end up tracked. A model path outside this install is cleared only when it no longer exists here: a model the user browsed to (a download, another drive) is kept. Two guards (`_eli_runtime_sanitize_path_string`, then `_eli_final_clean_path_value`) each cleared every outside model path, so a chosen model was gone from the settings at the next load (fixed in 2.5.21; 2.5.20 fixed only the first guard). The final pass now checks only that a model path leads somewhere: a relative one under `models/`, a full one that exists.
- **`settings_file_was_corrupt_on_last_load()`** — a settings file that exists but fails to parse (truncated write, disk-full, a concurrent-write race) silently fell back to `DEFAULTS` with no signal that the fallback happened, indistinguishable from a normal first run with no file yet. Anything rendering settings to the user should check this rather than presenting defaults as if they were read from disk.

Paths (`core/paths.py`, 808 LOC)

Dev-vs-packaged path resolution: `is_frozen` / `_is_dev_mode`, `_find_project_root`, then `data_dir`/`config_dir`/`cache_dir`/`m` `db_dir`/`artifacts_dir`/`user_db_path`/`agent_db_path`/`memory_db_path`. A source checkout uses project-local `artifacts/` + `config/`. The packaged app and an installed one both use v2's own root: `~/.local/share/ELI_v2` (`artifacts/`, `config/`, `cache/`), `%LOCALAPPDATA%\ELI_v2` on Windows, `~/Library/Application Support/ELI_v2` on macOS. One import surface (`get_paths`) so nothing hardcodes locations.

Nothing of v2's goes in a generic place: `~/.local/share/eli`, `~/.config/eli`, `~/.cache/eli`, `~/.eli/` and `~/.eli_*` belong to no single ELI version, and v3, the first version (`eli-ai`), ELI Legal and ELI VR keep their own folders. Until 2.5.15 an installed v2 used the generic `eli` folders and the packaged app kept its cache in `~/.cache/eli`; some files had a generic or relative fallback (`artifacts/...` lands in whatever folder ELI was started from). Every writer now goes through `get_paths`. A file v2 once kept in a generic place (the labs notebook and workspaces in `~/.eli/`, `~/.eli_user_id`) is copied into its own folders once by `adopt_legacy_file` (copied, not moved: another version may still use the old place). `tests/test_frozen_install_redistributability.py` scans the code for generic and relative paths.

Launchers are v2's too: `~/.local/bin/eli-v2-run`, `eli-v2*.desktop` menu entries and the `eli-v2` icon. Setup and the uninstaller replace or remove only v2's own entries (named `eli-v2…`, or an older entry whose contents name v2); another version's launchers are never touched.

STT VRAM awareness

`local_whisper_stt` puts faster-whisper on CUDA device 0 only when that card still holds it after the language model's plan: whisper's footprint on that card is measured on its first warm-up and kept (`data_dir/runtime/whisper_gpu.json`), its weights stand in until then. Any doubt keeps it on the processor. ctranslate2 is CUDA only, so other makers' cards keep whisper on the processor. `ELI_WHISPER_DEVICE` overrides. See `perception.md`.

Piper and LoRA device selection

Piper is a subprocess binary (`tts_piper/piper`), not onnxruntime-in-Python, and the shipped build is CPU-only for every vendor — no NVIDIA advantage to close there.

`eli/learning/lora_trainer._accelerator()` is the model for cross-vendor device selection generally: torch routes ROCm through the `torch.cuda` API (a HIP build answers `torch.cuda.is_available()`), so one code path serves NVIDIA and AMD, with the real vendor read from `torch.version.hip` and reported honestly rather than as "CUDA on a Radeon." Apple (mps) and Intel (xpu) are detected and honestly marked as unable to run that trainer. See `learning.md`.

Honest assessment

- **Strong:** loads are measured by llama.cpp on every device before they happen; every card of every vendor is read by one shared reader per source; the model's own chat template; env/settings/relative-path healing for moving between machines; a single broker + lock so concurrency is correct; a GPU pack that trusts a verified install.
- **Weak / watch:**
  1. The DXGI (Windows) and ioreg (macOS) readers are tested with fakes and on CI runners without GPUs, not yet on real Windows or Mac GPUs.
  2. **Settings sprawl** — `DEFAULTS` is large with several overlapping keys (`n_gpu_layers`/`gpu_layers`, `n_ctx`/`context_size`) and migration logic.
  3. Policy figures remain: the RAM budget (60%, 70% on processor-bound machines up to 16 GB, bounded 10-75%), the context target (0.9 of the trained context, half the memory left for the cache, a 2048 step) and the batch defaults.

4. `_portable_settings_for_storage` exists but isn't fully preventing personal values (e.g. `user_name`) from being persisted.
5. Install-time detection (`install.sh`, `setup/hardware_policy.py`) is still its own stack.